L'intégration et le déploiement continus (CI/CD) expliqués

CI/CD DevOps flux de travail du développeur automatisation déploiement

Le développement de logiciels modernes évolue rapidement, mais les processus manuels de test, de construction et de déploiement ne le font pas. L'intégration et le déploiement continus (CI/CD) automatisent ces flux de travail critiques, permettant aux équipes de livrer des fonctionnalités rapidement tout en maintenant la qualité et en minimisant les problèmes de production. Alors que l'intégration continue se concentre sur le test et la construction automatiques des modifications du code, le déploiement continu peut signifier soit la livraison continue (préparation du code pour la publication), soit le déploiement continu (publication automatique en production). Il est essentiel de comprendre cette distinction pour mettre en œuvre la bonne stratégie pour votre équipe.

Cet article vous guidera à travers les principes fondamentaux de l'intégration continue, partagera les meilleures pratiques éprouvées et explorera les techniques avancées qui peuvent transformer votre flux de développement, y compris la façon dont les plateformes modernes comme Upsun peuvent améliorer votre pipeline d'intégration continue avec des environnements de prévisualisation similaires à ceux de la production.

Qu'est-ce que l'intégration continue (IC) ?

Le sigle **CI** dans **CI/CD** signifie intégration continue, ce qui implique l'exécution automatique de tests et la construction du code sur un serveur distant à chaque fois que des changements sont apportés. Ce processus permet de s'assurer que le dernier commit de chaque branche passe toutes les vérifications et peut être publié ou fusionné en toute sécurité dans la branche principale.

Les plateformes courantes pour l'exécution de l'IC comprennent Jenkins, GitLab CI et GitHub Actions. Les exemples de cet article utilisent GitLab CI.

Le processus de base de l'IC comprend la vérification du style de code, l'exécution de différents types de tests et la construction du projet. Sur GitLab CI, un tel pipeline ressemblerait à ceci (en utilisant l'écosystème Node pour l'illustration) :

```
# .gitlab-ci.yml
stages: # les étapes sont exécutées dans l'ordre strict de la liste
  - build
  - test
  - deploy

build-code: # nom de la tâche
  stage: build
  script:
    - npm install # installer les dépendances
    - npm run build # construire le code

code-style:
  stage: test
  script:
    - npm install # installer les dépendances
    - npm run lint # vérifier le style du code

unit-tests:
  stage: test
```

```
script:
  - npm install # installer les dépendances
  - npm run test:unit # exécuter les tests unitaires
```

L'automatisation du processus de vérification du code permet de gagner du temps et d'améliorer la sécurité des versions en détectant les bugs à un stade précoce. Cependant, cela ne fait qu'effleurer la surface de ce que l'IC peut accomplir. Dans les projets de plus grande envergure, les pipelines de CI vont souvent bien au-delà de ces trois étapes.

La construction d'un pipeline de CI pour une base de code complexe peut inclure les éléments suivants :

```
unit-tests:
  stage: test
```

Les processus de construction complexes à plusieurs étapes nécessitent l'accomplissement de plusieurs actions. Par exemple, une architecture microservices peut impliquer la construction, le test et l'empaquetage indépendants de plusieurs services avant de les déployer ensemble.

Exécution parallèle et distribuée des tâches. Un pipeline CI pour un frontend et un backend monorepo peut exécuter en parallèle le linting, les tests unitaires et les vérifications de couverture de code pour chaque composant, réduisant ainsi le temps total de construction.

Constructions incrémentales. Dans les grands projets, le fait de ne reconstruire que les modules qui ont changé depuis le dernier commit peut accélérer considérablement les cycles d'itération, par exemple, dans un dépôt de moteur de jeu où les actifs et la logique de base évoluent indépendamment.

Optimisation des processus en termes de temps et de ressources. L'utilisation de configurations de matrices de tâches permet d'optimiser les constructions pour différents environnements (par exemple, tester une API sur les versions 16, 18 et 20 de Node.js) tout en minimisant les flux de travail redondants.

Gestion et mise en cache des dépendances : mise en cache efficace des dépendances importantes, telles que les couches Docker ou les paquets npm, afin d'éviter de les télécharger à chaque compilation, en particulier dans les environnements où la bande passante est limitée.

Intégration de différents cadres de test : assurer une exécution transparente de Jest pour les tests unitaires, Cypress ou Playwright pour les tests de bout en bout, et Postman pour les tests de contrats d'API.

Gérer les tests défaillants: mettre en œuvre des stratégies de réessai lors du test d'un système de messagerie en temps réel qui peut occasionnellement échouer en raison de conditions de concurrence.

Collecte et établissement de rapports sur les mesures. Visualisation des tendances en matière de taux de réussite de la construction, de couverture des tests et de durée d'exécution du pipeline via des tableaux de bord afin d'identifier rapidement les goulets d'étranglement ou les régressions.

Il ne s'agit là que de quelques exemples. Un pipeline de CI bien conçu peut être adapté pour répondre aux besoins spécifiques d'un projet, qu'il s'agisse d'améliorer les performances de construction, la qualité du code ou la collaboration entre les membres de l'équipe.

Qu'est-ce que la livraison continue (CD) ?

Une fois que le code a été construit et testé au cours de l'étape d'intégration, la livraison continue (CD) reprend la compilation de l'étape d'intégration et la prépare pour la mise en production. Il s'agit

notamment de déployer la version dans l'environnement de production ou de mise en scène, d'exécuter une autre série de tests de bout en bout et de télécharger le binaire dans un registre interne.

Les exemples présentés ici utilisent GitLab CI et l'écosystème Node, mais la même logique peut être mise en œuvre sur n'importe quelle plateforme CI/CD, comme Bitbucket Pipelines ou GitHub Actions.

Pour étendre l'exemple de code précédent avec le CD, vous devez ajouter des étapes de déploiement pour déployer la construction dans l'environnement de staging ou de production :

```
stages:
  - test
  - build
  - deploy-to-staging
  - deploy-to-production

deploy-to-staging:
  stage: deploy-to-staging
  script:
    - npm install # installer les dépendances
    - npm run deploy:staging # déployer en staging

deploy-to-production:
  stage: deploy-to-production
  when: manual # cette étape peut être démarrée manuellement uniquement, si le développeur souhaite déploye
  script:
    - npm install # installer les dépendances
    - npm run deploy:production # déployer en production
```

Les déploiements automatisés de base, comme l'étape "deploy-to-staging" dans l'exemple ci-dessus, constituent un bon point de départ pour les petits projets ou les cas d'utilisation simples. Cependant, les projets plus importants ou plus complexes nécessitent généralement des techniques supplémentaires pour améliorer la sécurité et le contrôle ; vous en apprendrez plus à ce sujet plus tard.

L'étape de déploiement vers la production dans l'exemple est conçue pour être manuelle plutôt qu'automatisée. Le choix entre les déploiements automatisés et manuels dépend de plusieurs facteurs, notamment la fréquence des versions souhaitées, la fiabilité des tests, les exigences réglementaires et les pratiques de gestion.

L'utilisation du déploiement continu et l'automatisation des processus de livraison permettent d'obtenir un retour d'information plus rapide sur la qualité des versions, ce qui favorise des versions moins risquées et plus prévisibles.

Qu'est-ce que le déploiement continu ?

Le déploiement continu est une forme de livraison continue dans laquelle toute modification du code qui passe toutes les vérifications est automatiquement déployée en production. Alors que la livraison continue met l'accent sur le fait que chaque modification est prête à être déployée, le déploiement continu va encore plus loin en supprimant complètement l'étape d'approbation manuelle.

Pour mettre en œuvre le déploiement continu à l'aide du pipeline présenté dans la section précédente, supprimez l'indicateur `when : manual` de l'étape de `déploiement vers la production`. Si vous n'avez pas besoin de déployer vers l'environnement staging (en supposant que vous avez confiance dans les tests CI comme principale sauvegarde de la qualité du code), vous pouvez également omettre l'étape `deploy-to-staging` :

```
deploy-to-production:
  stage: deploy-to-production
  script:
    - npm install # installer les dépendances
    - npm run deploy:production # déployer en production
```

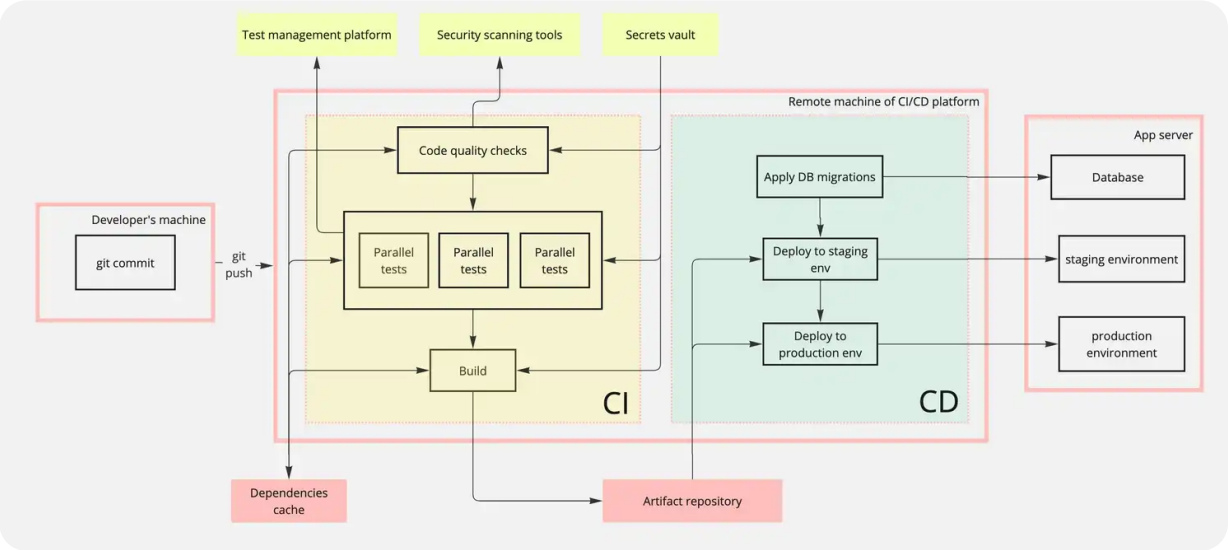
Le déploiement continu offre les mêmes avantages que la livraison continue : réduction des efforts manuels, amélioration de la cohérence des versions et capacité à fournir des mises à jour de haute qualité en continu. Cependant, il nécessite des tests et une surveillance automatisés robustes, car les modifications de code sont déployées sans examen manuel, ce qui augmente le risque de problèmes en production. Elle exige également un investissement important dans l'infrastructure et les processus afin de garantir la fiabilité et de permettre des retours en arrière rapides en cas de besoin.

Note*: Bien que l'acronyme "CD" soit utilisé pour désigner à la fois la livraison continue et le déploiement continu, la suite de cet article utilisera le terme "CD" pour désigner le déploiement continu.*

Architecture du pipeline CI/CD (pour une vue d'ensemble)

Voici comment l'intégration et le déploiement continus fonctionnent généralement dans un pipeline DevOps :

1. **Poussée de code**: Le développeur envoie les modifications de code à son dépôt VCS, GitLab, GitHub, Bitbucket, etc.
2. **Construction et test**: L'IC exécute automatiquement des tests et construit le code sur la plateforme CI/CD.
3. **Déploiement**: Si l'étape CI passe toutes les vérifications, CD déploie le code dans des environnements de staging ou de production.
4. **Application en direct**: L'application mise à jour est maintenant en ligne et accessible aux utilisateurs.



Comme vous pouvez le constater, un pipeline complet implique de nombreux systèmes :

Dans les coulisses, chaque poussée de code se déclenche :

- L'analyse de sécurité et les contrôles de qualité s'exécutent automatiquement
- Les tests s'exécutent en parallèle pour minimiser le temps de construction
- Les données sensibles (clés d'API, informations d'identification) sont injectées en toute sécurité via la gestion des secrets.
- Les dépendances sont mises en cache afin d'accélérer les constructions ultérieures.
- Les migrations de bases de données s'appliquent automatiquement pendant le déploiement

L'exécution du pipeline implique plusieurs processus parallèles :

- Les outils de sécurité statique recherchent les vulnérabilités dans les changements de code.
- Les tests sont exécutés en parallèle pour accélérer l'exécution.
- Les résultats des tests sont envoyés à une plateforme de gestion des tests.
- Une fois que l'artefact de construction final est généré, il est stocké dans le référentiel d'artefacts et déployé dans les environnements appropriés.

Il ne s'agit là que d'un exemple de pipeline ; d'autres composants peuvent être nécessaires en fonction des exigences de votre application.

Techniques CI/CD avancées

Comme nous l'avons mentionné, si vous avez un petit projet, un déploiement automatisé de base, comme l'exemple utilisé précédemment, serait un bon point de départ. Cependant, les projets plus importants ou plus complexes nécessitent généralement des techniques supplémentaires pour améliorer la sécurité et le contrôle.

L'une des approches consiste à utiliser des indicateurs de fonctionnalités pour déployer du code avec des fonctionnalités qui sont cachées aux utilisateurs jusqu'à ce qu'elles soient considérées comme stables, ce qui permet des versions incrémentielles et un retour en arrière facile si nécessaire. Il existe différents types de drapeaux de fonctionnalités, tels que les "release toggles", qui permettent un déploiement progressif des fonctionnalités, et les "experiment flags", utilisés pour les tests A/B. Par exemple, un site de commerce électronique peut développer un nouveau filtre de recherche, mais le garder caché derrière un drapeau de fonctionnalités jusqu'à ce que des tests approfondis soient terminés, de sorte qu'il puisse être activé instantanément lorsqu'il est prêt.

Vous pouvez également utiliser des "versions canaris" et des retours en arrière pour atténuer les risques d'introduction de bugs, de problèmes de performance ou d'impacts involontaires sur les utilisateurs lors du déploiement de nouvelles fonctionnalités. Une "version canari" consiste à déployer d'abord la fonctionnalité auprès d'un petit segment d'utilisateurs. Vous pouvez alors collecter des mesures et vous assurer que la fonctionnalité fonctionne comme prévu. En cas de problème, vous pouvez revenir en arrière pour ce petit groupe, ce qui rend le processus plus rapide et moins risqué. Vous pouvez aller plus loin en automatisant les retours en arrière sur la base des indicateurs que vous avez choisis précédemment. L'automatisation vous permet également d'éviter de gérer manuellement les migrations de bases de données en cas de retour en arrière. Ces techniques peuvent vous aider à effectuer des déploiements progressifs et sans heurts pour vos utilisateurs.

Les outils de surveillance et de retour d'information sont essentiels pour obtenir une vue

Les outils de surveillance et de retour d'information sont essentiels pour obtenir une vue d'ensemble de vos services, garantissant ainsi un déploiement sûr et permettant des retours en arrière rapides en cas de problèmes. Des outils tels que Blackfire.io sont des compléments précieux au processus de mise en production. Les mesures clés à surveiller comprennent les taux d'erreur, la latence des requêtes, la progression du déploiement (en particulier pour les déploiements canari), les taux de réussite et d'échec des drapeaux de fonctionnalités et les mesures d'engagement des utilisateurs. Ces informations permettent d'identifier les problèmes, tels que l'augmentation du nombre d'erreurs ou l'abandon par les utilisateurs, ce qui incite à revenir en arrière si nécessaire. Les outils de surveillance permettent également de configurer des alertes et des tableaux de bord spécifiques aux déploiements pour suivre les paramètres critiques, améliorant ainsi l'observabilité et contribuant à la publication sûre et efficace des mises à jour.

Meilleures pratiques pour la mise en œuvre de CI/CD

Il existe également des bonnes pratiques à suivre pour améliorer les flux de travail tout en renforçant la fiabilité et la sécurité du processus CI/CD.

Automatisation des tests

L'automatisation des tests dans le cadre du processus CI/CD garantit que les changements sont entièrement testés sans intervention manuelle, ce qui permet de gagner du temps et de réduire les risques. En suivant la pyramide des tests, les tests unitaires à la base, les tests d'intégration au milieu et les tests de bout en bout (E2E) au sommet, la couverture et la vitesse sont équilibrées, ce qui garantit que les problèmes sont détectés au bon niveau sans surcharger le pipeline.

Des éléments clés, tels que la couverture du code, les tests de mutation et la gestion des données, améliorent encore les tests CI/CD. La couverture du code permet de vérifier que les domaines essentiels sont testés, tandis que les tests de mutation identifient les points faibles en vérifiant si les tests détectent les erreurs intentionnelles. La gestion des données de test par le biais de mocks ou de snapshots renforce la fiabilité et la répétabilité, améliorant ainsi la qualité globale du logiciel. Pour plus d'efficacité, l'exécution des tests en parallèle, l'exécution sélective des tests impactés et l'isolation des environnements permettent d'optimiser la durée et la stabilité des tests.

Optimisations

L'optimisation des pipelines CI/CD est cruciale pour minimiser le temps d'attente des développeurs et, par conséquent, pour déployer les changements plus rapidement. En outre, l'optimisation de la durée des pipelines est essentielle pour contrôler les coûts, car la plupart des plateformes facturent en fonction du temps d'utilisation et des ressources consommées. Voici trois exemples de la manière dont vous pouvez optimiser votre pipeline :

Mise en cache des dépendances. En stockant et en réutilisant des bibliothèques ou des modules précédemment téléchargés, vous pouvez éviter les installations redondantes, ce qui permet de gagner du temps et d'économiser des minutes facturables lors de l'exécution du pipeline.

Exécuter les tâches en parallèle. Cela permet à plusieurs vérifications de s'exécuter simultanément, ce qui réduit considérablement les temps de construction et de test. Par exemple, un grand ensemble de tests de bout en bout peut être divisé en sous-ensembles plus petits et exécuté en parallèle, ce qui accélère considérablement le processus.

Optimiser les suites de tests. Exécutez les tests critiques en premier afin de détecter rapidement les défaillances et de gagner du temps en n'exécutant pas d'autres tâches en cas d'échec des tests. L'une des techniques d'optimisation les plus efficaces consiste à mettre en œuvre un mécanisme permettant de n'exécuter que les tests affectés par des modifications récentes.

D'autres optimisations comprennent l'optimisation des images Docker et la gestion efficace des dépendances, à la fois au sein du travail de CI et de ceux utilisés pour la construction.

Sécurité

Les processus CI/CD ont des exigences de sécurité distinctes si vous voulez vous assurer que le pipeline et les applications restent protégés contre les vulnérabilités. Il s'agit notamment d'adopter une approche multidimensionnelle pour identifier les vulnérabilités à différents stades du développement et du déploiement. Les tests statiques de sécurité des applications (SAST) analysent le code source ou les binaires pour identifier les vulnérabilités de sécurité dès le début du développement. Les tests dynamiques de sécurité des applications (DAST) analysent les applications en cours d'exécution pour détecter des problèmes tels que les attaques par injection et les défauts de configuration. L'analyse de la composition des logiciels (SCA) examine les dépendances et les bibliothèques tierces pour détecter les vulnérabilités connues et vérifier la conformité des licences. Ces outils travaillent ensemble pour sécuriser les applications tout au long du cycle de développement et de déploiement.

L'exemple de pipeline ci-dessus utilise des outils de sécurité statiques dans l'étape de vérification de la qualité du code pour rechercher les vulnérabilités dans le code. Pour mettre en œuvre des tests de sécurité dynamiques dans une application en cours d'exécution, vous pouvez configurer une tâche supplémentaire qui attend que la compilation soit générée, la pousse vers un environnement de prévisualisation (éventuellement un environnement hébergé sur Upsun), puis exécute un outil DAST (comme le Zap de l'OWASP) pour exécuter des contrôles de sécurité sur elle.

Les données sensibles, comme les clés d'API, doivent être stockées et injectées dans les pipelines de manière sécurisée. Vous pouvez utiliser la gestion des secrets intégrée à votre plateforme CI/CD pour chiffrer les données stockées et les injecter en toute sécurité dans le code du pipeline. Si vous avez des exigences avancées, telles que la génération dynamique de clés ou un système de permissions flexible, vous pouvez utiliser des services externes, tels que HashiCorp Vault ou AWS Secrets Manager.

Si votre pipeline publie des artefacts, tels que des builds ou des résultats de tests, il est important de les protéger contre tout accès non autorisé afin que personne ne puisse accéder aux informations privées générées par votre pipeline. Vous pouvez également ajouter une étape d'analyse de la vulnérabilité des artefacts pour vous assurer qu'aucune donnée sensible n'a été accidentellement injectée et qu'ils ne contiennent pas de code potentiellement dangereux.

Suite

Cet article a exploré les concepts fondamentaux de CI/CD, les meilleures pratiques de mise en œuvre et les techniques avancées qui peuvent transformer votre processus de mise en production. Les principaux avantages de la CI/CD sont une mise sur le marché plus rapide, une fiabilité accrue, une réduction des erreurs manuelles et la possibilité d'adapter les flux de développement à des projets complexes.

Mais voici le défi: même avec de solides pipelines CI/CD, les tests dans des environnements similaires à la production restent un goulot d'étranglement. La plupart des équipes sont confrontées à des incohérences au niveau de l'environnement, à des ressources de mise en scène limitées et à l'incapacité de tester avec des données réelles en toute sécurité.

C'est là qu'Upsun améliore vos flux de travail CI/CD existants. Plutôt que de remplacer votre plateforme CI/CD, Upsun s'intègre de manière transparente avec GitHub Actions, GitLab CI, Jenkins et d'autres outils pour résoudre les défis de déploiement et de test que les pipelines seuls ne peuvent pas résoudre.

Au lieu de gérer une infrastructure de déploiement complexe, votre pipeline CI/CD peut se concentrer sur ce qu'il fait de mieux, construire et tester du code, tandis qu'Upsun s'occupe du déploiement, du provisionnement de l'environnement et de l'hébergement de niveau production.